

altairsim — The SD Systems SBC-200 -- a Z80 single-board computer

2026-07-26 · fcd24c0

The **SD Systems SBC-200** was a whole computer on one S-100 board: a 4 MHz Z80, an Intel 8251 serial console, a Z80-CTC baud generator, a parallel port, RAM, and boot-PROM sockets, acting as the system's bus master. It shipped with the **SD/MS monitor** in EPROM and, with a VersaFloppy controller, booted SDOS or CP/M.

```
cd examples/sdsys
altairsim sbc200.toml
(prompt Enter)  -> .
```

This is a thin delta on the built-in `sbc200` machine (`altairsim sbc200`): the 4 MHz Z80, the 8251 console at ports 7C/7D, the **MSMONR21** monitor in EPROM at E000, the SD **DDBIOS** disk BIOS at F000, and RAM filling the rest of the 64K.

Press Enter to get the prompt

The monitor prints nothing until you press Enter, and that is authentic. The SBC-200 wires the 8251's receive-data line to its /DSR input, and MSMONR21 uses that to **auto-detect your baud rate**: on reset it waits, then times the start bit of the first character you type (a carriage return), reading status bit 7 in a tight loop. Press Enter and it matches your speed and prints its `.` prompt.

The simulator models that line in emulated processor time, so the genuine ROM auto-bauds on the console with no changes. (If a run ever seems to "hang" at startup, it is waiting for that first CR -- press Enter.)

The monitor

At the `.` prompt, MSMONR21 gives you a full Z80 monitor -- memory display/edit, fill, move, search, I/O port access, breakpoints and single-step, and hex arithmetic. A few to try:

```
.D E000 E01F      display the monitor ROM, hex + ASCII
.E 8000           examine/substitute memory at 8000
.H 1234 0100      hex arithmetic: sum then difference
```

Type `.` to abort a command back to the prompt. The full command set is in `reference/SD Systems Monitor.md`.

Booting SDOS

`sdos.toml` adds a **VersaFloppy II** floppy controller with a bootable SDOS master disk in drive A. From the monitor prompt, `C` cold-boots the OS:

```
cd examples/sdsys
altairsim sdos.toml
(prompt Enter)  -> .
C              -> cold-boot SDOS from drive A
```

```
32K SD-OS Version 1.8B
DELTEC ENTERPRISES LLC
```

```
[A]
```

`[A]` is the SDOS prompt. The monitor's other disk commands work too: `R/W` read and write 128/256-byte sectors, `Z` formats a diskette. The disk is `SDOS-18B-SSDDR-256-32K-MASTER.DSK` -- an 8" single-sided, double-density, 256-byte-sector image (26 sectors x 77 tracks), sysgen'd for a 32K system. It is mounted **read/write**, so SDOS can save files; run `git checkout` on it to restore the master if you change it.

Booting CP/M 2.2

`cpm.toml` boots **SD Systems CP/M 2.2** from the same VersaFloppy II, and it exercises two pieces of the SBC-200 that SDOS does not:

```
cd examples/sdsys
altairsim cpm.toml
(prompt Enter)  -> .
C              -> cold-boot CP/M from drive A
```

```
64k CP/M vers 2.2 for MS-610
COMPUTING INFORMATION SCIENCES
```

```
A>
```

`A>` is the CP/M prompt -- type `DIR` for the directory. Two things make this different from the SDOS machine:

- **The keyboard is interrupt-driven.** The CP/M CBIOS console driver takes input *only* through a Z80 mode-2 vectored interrupt: the 8251's RxRDY triggers the SBC's Z80-CTC channel 1, which vectors (byte `0x82`) to the keyboard handler. SDOS polled the keyboard, so it booted without interrupts; CP/M will not. Everything you type at `A>` arrives that way.
- **The onboard PROM switches out.** This is a full **64K** machine whose monitor (E000) and DDBIOS (F000) live in the SBC's onboard boot-PROM sockets, shadowing RAM. When CP/M's cold boot has loaded the system high, it does `OUT 7F,3` to drop the PROM out of the map, and the 64K of RAM under it becomes CP/M's. (`sbc200.toml` / `sdos.toml` , by contrast, keep their ROMs on the memory card and never switch anything out.)

The disk is `SD-CPM22R4-SSDDR-256-64K.DSK` -- the same 8" DD-256 format as the SDOS master, sysgen'd for a 64K system -- mounted **read/write**. A 32K image (`SD-CPM22R4-SSDDR-256-32K.DSK`) boots on the same machine too; being a 32K system it loads below the PROM and so never needs the switch-out.

A video console instead of a serial terminal

`sbc200v.toml` runs the same machine with the **SD Systems VDB-8024** video board as its console instead of the 8251 serial port. It boots the video build of the monitor (**SDMONV21**), which prints its `.` prompt straight onto an 80x24 screen -- and with no "press Enter first", because the VDB is a parallel-handshake terminal, not a serial line with a baud rate to measure.

```
cd examples/sdsys
altairsim sbc200v.toml
.                <- the monitor prompt, ON THE VIDEO SCREEN
```

With SDL3 the screen is a window in the board's own character font; window and terminal keystrokes both reach the monitor. The command set is identical to the serial machine's. See `reference/SD Systems VDB-8024.md` for the board.

`sdosv.toml` boots **SDOS on the video console** -- the video twin of `sdos.toml` :

```
cd examples/sdsys
altairsim sdosv.toml
.                <- the monitor prompt, ON THE VIDEO SCREEN
C                <- cold-boot SDOS from drive A
[A]             <- SDOS is live; type at the video window
```

The video keyboard is interrupt-driven, and that is the whole trick. The monitor *polls* the VDB keyboard, so it boots and takes the `C` command with no interrupt at all -- but SDOS does not poll: its video CBIOS runs console input under a Z80 **mode-2 interrupt**. The VDB's

keyboard strobe is strapped to S-100 **VI2** (`interrupt = "vi2"` on the board), the SBC-200's **Z80-CTC** turns that into the mode-2 vector `0x02` , and the CBIOS keyboard ISR reads the byte. Without it the monitor works but a booted OS never sees a key -- which is exactly the same interrupt path SD CP/M's *serial* console uses (vector `0x82`), one board over.

What is not here yet

The serial console, the video console, the VersaFloppy disk, the SDOS/CP/M keyboard interrupt (on both the 8251 and the VDB-8024) and the SBC-200 memory switch-out all work today. The Z80-CTC is modeled only as far as those keyboard interrupts need (its baud-generator and timer channels are not observable at flat-out speed), and the reset auto-start jam is still stood in for by `startup = ["RUN E000"]` rather than the authentic PROM-at-the-reset-vector with its `IN 7F` release.